

Title

A Multi-agent AI System for Automating Tokamak-Plasma Simulation for Nuclear Fusion Energy.

Presenters

- Sarthak Sharma <ss694@buffalo.edu>, PhD candidate in Computational and Data Sciences, State University of New York at Buffalo, 0009-0009-6746-169X.
- Junchao Zhang <jczhang@anl.gov>, Division of Mathematics and Computer Science, Argonne National Laboratory, 0000-0003-0367-2358.
- Lois Curfman McInnes <curfman@anl.gov>, Division of Mathematics and Computer Science, Argonne National Laboratory, 0000-0002-6381-4736.

Keywords

AI agents; scientific computing; research software engineering; multi-agent code generation; verification and validation; reproducibility and provenance; guardrails; human in the loop; PETSc; tokamak; Grad-Shafranov equilibrium; plasma physics; nuclear fusion energy; clean energy; artificial intelligence; large language models; Claude.

Abstract

Building a correct, performant HPC simulation from a scientific idea demands scarce, largely implicit expertise in numerical methods and library APIs. AI agents can now draft such code, but a draft that *looks* right is not necessarily one that an engineer can *trust*. We take a research-software-engineering (RSE) view of a real fusion-energy problem: not whether AI can replace that expertise, but what an RSE must build around a multi-agent AI system (orchestration, verification gates, provenance, and guardrails) to make its output trustworthy.

Using a multi-agent AI system [3], from a plain-language prompt, we generate a PETSc simulation of the tokamak **Grad-Shafranov equilibrium** for nuclear fusion energy. In this poster, we will present the PETSc multi-agent system and demonstrate its application on this problem. The system includes a mathematical modeling agent, a numerical analysis agent, and an HPC code generation and execution agent. Around them, there is a project-owned orchestration driver that records every artifact (model, discretization decision, generated source code, build / run logs) with full provenance, so that runs are reproducible, resumable, and auditable. There is also a verification harness that gates every result before it is accepted.

From the prompt (*“Grad-Shafranov equilibrium for magnetically confined plasma in a tokamak”*), the mathematical modeling agent identified the governing equation and returned its strong and weak forms; the numerical analysis agent selected a nonlinear solver on a structured grid; and the HPC code generation and execution agent produced a compact PETSc program that discretizes the operator on a distributed array with a true Jacobian matrix, and embeds a method-of-manufactured-solutions check. We do not treat “it compiled and ran” as sufficient evidence of success: an MPI-parallel grid-refinement study confirms second-order convergence. Verification, not the model’s confidence, decides acceptance. The same pipeline then produced one parameterized solver for several real-machine-shaped equilibria. Each was verified against an exact analytic benchmark and cross-checked against an independent community code. We scaled from the toy anchor to trustworthy shaped cases *without re-architecting the generated solver*.

Running the AI agents on a shared HPC machine took environment and dependency wrangling, working-directory-independent process spawning, and graceful degradation when optional components were absent. There are also **guardrails**: in one instructive failure, a hard-coded iteration cap in the built-in orchestrator mislabeled a genuinely successful multi-stage run as a failure. We diagnosed it and contributed the fix upstream. Human effort is not eliminated but relocated, from writing solver code to engineering the orchestration, verification, and guardrails that make an AI system’s output trustworthy and its failures legible.

We report per-run **decision-gate metrics**: correctness (model identified, program compiled and ran, convergence order), efficiency (execution duration, LLM / tool calls), and where human effort went (as a lightweight, reusable way to *evaluate* an agentic workflow rather than merely admire its output). The contribution is a candid, verification-driven account of how a research software engineer can wrap multi-agent AI code generation so that it produces trustworthy HPC simulation code for a flagship fusion-energy problem, and of the guardrails needed when it does not.

In future work, we will raise the simulation difficulty to nonlinear profiles, time-dependent and resistive magnetohydrodynamics, and three-dimensional tokamak plasma, to map where the system succeeds and where it breaks, and to design guardrails and human-in-the-loop checkpoints that keep engineers in control as problems get harder.

References

1. B Smith, H Zhang, J Zhang, S Balay, L Chen, M Keçeli, L C McInnes. *Improving usability and productivity of PETSc with agent-based workflows*. 2026. doi:10.13140/RG.2.2.35234.80326.
2. S Balay et al. *PETSc/TAO Users Manual*. ANL-21/39 Revision 3.25, Argonne National Laboratory, 2026. <https://petsc.org>.
3. PETSc multi-agent MCP servers. https://gitlab.com/petsc/petsc_mcp_servers; <https://mcp.petsc-ai.org>.
4. H Grad, H Rubin. *Hydromagnetic equilibria and force-free fields*. Proceedings of the Second UN Conference on the Peaceful Uses of Atomic Energy, 1958. (Grad–Shafranov equation.)
5. A J Cerfon, J P Freidberg. “One size fits all” analytic solutions to the Grad–Shafranov equation. *Physics of Plasmas* 17, 032502, 2010.
6. This work: https://github.com/engineer-scientist/petsc_mcp_servers_tokamak.

Connection to Mission, Goals, and Interests of the US-RSE Community

Research software engineers increasingly sit between AI code-generation tools and the trusted, performance-critical libraries that underpin computational science. This submission speaks directly to that intersection. It is a candid RSE case study of *wrapping* an agentic AI system around a real HPC library (PETSc: Portable, Extensible Toolkit for Scientific Computing) for a real DOE-mission problem (fusion-energy plasma), rather than a toy benchmark. It includes the unglamorous engineering that makes such systems actually work on shared machines (environment and dependency wrangling, working-directory-independent process spawning, graceful degradation when optional components are absent, and reproducible, provenance-tracked runs), and the guardrails needed to detect and manage cases in which an agent silently produces incorrect results. Our theme is that AI does not replace the research software engineer; instead, it relocates the RSE’s work to orchestration, verification, and guardrails. We treat **verification as a first-class deliverable** (in testing, generated code was accepted only after it passed an exact-solution and grid-convergence gate), reflecting the community’s emphasis on correctness, testing, and reproducibility over raw output. We hope that the poster sparks discussion about how RSEs can responsibly adopt, evaluate, and harden AI-assisted workflows for scientific software.